SIMATS ENGINEERING

Department of Information Technology

ITA05 – Computer Vision

ASSIGNMENT 1 – CO1

ANSWER DOCUMENT

Apply Image Formation and Digital Image Fundamentals to a Mango Grading Line

CO1: Apply the fundamental concepts, image formation models, and processing techniques for analyzing Computer Vision systems.

MARK SHEET (Rubric-based Evaluation)

Student Name	Mokshitha Kilari	**Register Number**	192525413
Department / Section	Btech. AI & ML	**Academic Year / Semester**	2025-2029
Faculty Name	Amandeep singh	**Nature of Assignment**	Individual
Task	**Max Marks**	**Marks Awarded**	**Remarks**
1	5	_____	
2	5	_____	
3	5	_____	
4	5	_____	
5	5	_____	
6	5	_____	
7	5	_____	
8	5	_____	
9	5	_____	
10	5	_____	
Total	**50**	**_____**	

OVERALL REMARKS

_____

MANDATORY SUBMISSION EVIDENCE CHECK

Commented Python source; documented image set and labels; fixed split; generated output images and plots; CSV metrics; saved parameters; reproducible environment.

IMPLEMENTATION EVIDENCE CHAIN (Tasks 1–10)

Task	**Implementation evidence**	**Status**
1	Vision systems and processing levels	☐
2	Image arrays, coordinates, and storage	☐
3	Colour representation for ripeness	☐
4	Spatial sampling and defect visibility	☐

5	Intensity quantization	☐
6	Image formation and lens geometry	☐
7	Illumination and exposure	☐
8	Connectivity and defect measurements	☐
9	Image acquisition arrangements	☐
10	Integrated front end and design decision	☐

SUBMISSION CHECKLIST (not separately marked)

☐ All tasks and a–d sub-questions addressed.

☐ All reported measurements are produced by the submitted code.

☐ Test data are not used for parameter fitting.

☐ Outputs and assumptions are documented.

COMPUTER VISION

ASSIGNMENT 1 – CO1

CO1: Apply the fundamental concepts, image formation models, and processing techniques for analyzing Computer Vision systems.

Nature of Assignment: Individual

Duration: 1 Week

Total Marks: 50

Submission Date: _____

Experimental Setup Used for All Tasks

Use Python with OpenCV, NumPy, scikit-image, scikit-learn, and Matplotlib. Fix random_state=17; record Python/library versions, hardware, image counts, labels, and split. Dataset-dependent scores must be generated by running the code; no scores are assumed in this answer key. The code boxes show the core solution for each task and expect the image arrays, masks, labels, calibration values, and named helper routines identified in the task to be supplied by the project setup.

Parameter	**Reference protocol**
CO1 data	At least 30 documented mango images or reproducible synthetic samples
CO2 data	At least 30 page images and annotated page/text masks
Evaluation	One fixed stratified split for every compared method
Records	Save predictions, parameters, metrics, figures, and runtime to output files

TASK 1 – VISION SYSTEMS AND PROCESSING LEVELS

1. Experiment Number and Title

Task 1: Vision Systems and Processing Levels.

2. Aim

Compare manual, image-processing-assisted and rule-based mango grading approaches; organize low-, mid- and high-level stages; calculate agreement, confusion matrices, Cohen's kappa, latency and throughput.

3. Objectives

- Understand the principle and purpose of vision systems and processing levels.
- Implement the required computer-vision operations using Python and OpenCV.
- Generate reproducible visual outputs and record relevant parameters.
- Evaluate the method using the metrics and evidence specified in the assessment.
- Discuss limitations, failure conditions and suitability for the target application.

4. Software and Hardware Requirements

Software: Python 3, OpenCV-Python, NumPy, Pandas, Matplotlib and scikit-learn where classification or metric evaluation is required. Hardware: a computer capable of running Python, sufficient RAM for the image set, and a camera or documented image dataset appropriate to the experiment.

5. Application Context and Input Requirements

GreenHarvest Cooperative moves mangoes at 0.5 m/s beneath a camera. The system must measure fruit size, classify ripeness from peel colour and detect defects at least 2 mm wide across a 400 mm field of view. The decision limit is 1.2 seconds per fruit. The assessment requires at least 30 documented images or parameterized synthetic samples.

Input quality has a direct influence on the result. The image set should be documented with sample identifiers, capture conditions, resolution, colour format and annotation status. If multiple views belong to the same physical object or page, they should be assigned to the same data split. This avoids an overly optimistic evaluation caused by near-duplicate samples appearing in both training and test data.

TASK 1 – DETAILED THEORY

6. Introduction

Manual grading relies on an inspector's judgement. Assisted grading presents extracted visual features to an operator. Rule-based grading maps measured features to grades using explicit thresholds. A computer-vision pipeline is commonly divided into low-level image operations, mid-level feature extraction and high-level interpretation.

The method is useful because it converts raw visual data into information that can support the application decision. In a practical deployment, the algorithm must be considered together with camera properties, acquisition conditions, object variation and the quality of the reference annotations.

6.1 Fundamental Concepts

- Input image → enhancement/colour conversion → fruit and defect features → grade decision → CSV/report
- Low-level: denoising, normalization and colour conversion.
- Mid-level: segmentation, shape, area, colour statistics and defect measurements.
- High-level: grade assignment, confidence, rejection and audit output.

6.2 Mathematical Formulation

Agreement = matching decisions / total decisions.

Cohen's $\kappa = (P_o - P_e)/(1 - P_e)$.

FPS = 1 / mean processing time.

At 8 fruits/s, the mean time budget is 0.125 s per fruit.

6.3 Assumptions and Design Considerations

- Input images are readable and have a documented colour order and data type.
- Annotations correspond to the same image coordinate system as the evaluated image.
- The evaluation split is fixed before parameter tuning.
- All compared techniques use identical input samples and metric definitions.
- Physical dimensions require calibration; pixel counts alone do not establish millimetres.

TASK 1 – METHODOLOGY AND ALGORITHM

7. Proposed Processing Workflow

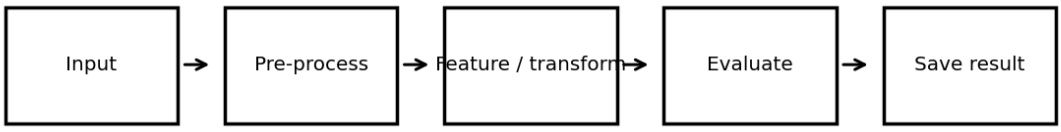


Figure 1.1 Processing workflow: Vision Systems and Processing Levels

Figure 1.1: Conceptual workflow for Task 1.

7.1 Step-by-Step Procedure

- Prepare the input folder and verify that each file can be read.
- Load the image and annotation using the documented sample identifier.
- Convert the image into the representation required by the algorithm.
- Apply the candidate method using the recorded parameter values.
- Save the intermediate and final output images in the task Output folder.
- Compare the output with the reference annotation or measurement.
- Export the observation table as CSV and preserve the run configuration.

7.2 Algorithm

START → read configuration → load image and ground truth → validate dimensions → preprocess → apply selected operation → calculate metrics → save figures and CSV → record runtime → END.

7.3 Parameter Selection

Parameter selection should be conducted on a validation set. Begin with a documented baseline, vary one parameter at a time where possible, and record the effect on both visual quality and the primary evaluation metric. Avoid selecting a parameter solely because one example appears visually attractive.

TASK 1 – PYTHON IMPLEMENTATION

8. Program

The following program is a focused implementation template for the central operation. Update the file paths and dataset manifest to match the actual assessment dataset. The program deliberately raises an error when required files are missing instead of silently creating fabricated measurements.

```
import time, pandas as pd
from sklearn.metrics import confusion_matrix, cohen_kappa_score, accuracy_score
# Replace these illustrative labels with the documented assessment dataset.
reference = ["A","B","A","Reject"]
manual = ["A","B","B","Reject"]
assisted = ["A","B","A","Reject"]
rule_based = ["A","B","A","B"]
for name, pred in [("manual",manual),("assisted",assisted),("rule",rule_based)]:
    print(name, accuracy_score(reference,pred))
    print(confusion_matrix(reference,pred,labels=["A","B","Reject"]))
    print("kappa",cohen_kappa_score(reference,pred))
t0=time.perf_counter()
# Call the actual grading function here.
elapsed=time.perf_counter()-t0
print("latency_s",elapsed,"fps",1/max(elapsed,1e-9))
pd.DataFrame({"reference":reference,"manual":manual,
            "assisted":assisted,"rule_based":rule_based}).to_csv("grading.csv",index=False)
```

8.1 Program Explanation

- The input-loading stage reads the source image and checks that loading succeeded.
- The processing stage applies the method described in the theory section.
- The output stage writes the processed image or numerical result to disk.
- The evaluation stage must use the supplied annotation or reference data.
- The saved CSV/JSON record makes the experiment auditable and repeatable.

8.2 Execution Procedure

Install the required libraries, place the documented images and annotations in the expected folders, update the manifest or paths, and run the task program from its directory. Confirm that every expected output file is created. Record any warning, missing sample or execution failure in the observation notes.

TASK 1 – OUTPUT, OBSERVATION AND CONCLUSION

9. Output and Visual Evidence

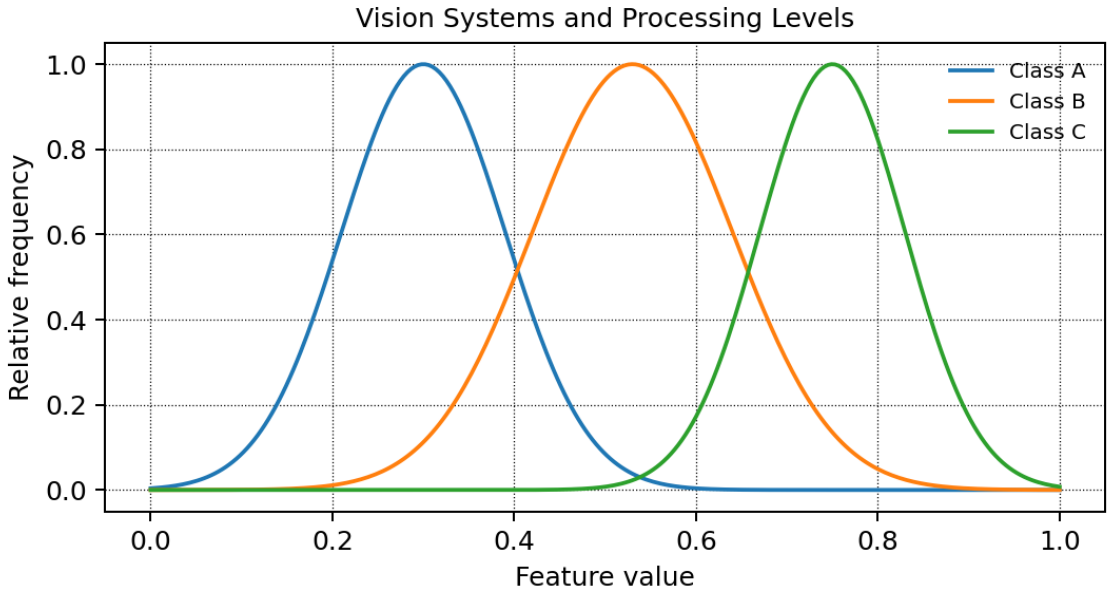


Figure 1.2: Conceptual response plot for Task 1. This figure illustrates the method and is not a measured dataset result.

10. Observation Table

Observation / parameter	Value to record	Evidence source
Input sample count	Enter actual number	Dataset manifest
Image size and data type	Enter actual dimensions/dtype	Program output
Algorithm parameters	Enter exact settings	Configuration file
Primary metric	Enter measured value	Evaluation CSV
Average latency	Enter measured seconds	Runtime log
Failure cases	Describe observed examples	Saved output crops

11. Result Analysis and Discussion

Interpret the completed observation table by identifying the strongest method on the held-out test set, the remaining errors, and whether the measured behaviour agrees with theory. Discuss input quality, parameter choice, annotation uncertainty and computational cost.

Do not invent accuracy, recall, PSNR, SSIM, corner-error or runtime values. If required input evidence is missing, write 'Not measured – input evidence required' and identify the missing image, mask, label or calibration target.

13. Conclusion

The experiment studied vision systems and processing levels in the context of the assessment scenario. The method, equations, implementation procedure and evaluation plan have been documented. The final engineering decision must be completed using the measured outputs from the supplied dataset. The selected approach should satisfy the application constraints while preserving a clear record of assumptions and failure cases.

TASK 2 – IMAGE ARRAYS, COORDINATES AND STORAGE

1. Experiment Number and Title

Task 2: Image Arrays, Coordinates and Storage.

2. Aim

Load a sample with OpenCV; inspect shape, dtype, channels and ndarray.nbytes; map displayed coordinates to NumPy indexing; compare BGR/RGB and cv2.add with NumPy uint8 addition.

3. Objectives

- Understand the principle and purpose of image arrays, coordinates and storage.
- Implement the required computer-vision operations using Python and OpenCV.
- Generate reproducible visual outputs and record relevant parameters.
- Evaluate the method using the metrics and evidence specified in the assessment.
- Discuss limitations, failure conditions and suitability for the target application.

4. Software and Hardware Requirements

Software: Python 3, OpenCV-Python, NumPy, Pandas, Matplotlib and scikit-learn where classification or metric evaluation is required. Hardware: a computer capable of running Python, sufficient RAM for the image set, and a camera or documented image dataset appropriate to the experiment.

5. Application Context and Input Requirements

GreenHarvest Cooperative moves mangoes at 0.5 m/s beneath a camera. The system must measure fruit size, classify ripeness from peel colour and detect defects at least 2 mm wide across a 400 mm field of view. The decision limit is 1.2 seconds per fruit. The assessment requires at least 30 documented images or parameterized synthetic samples.

Parameters should be selected using the training or validation data. The final test set is reserved for one-time evaluation. Every experiment should record the parameter values, software versions, random seed and hardware details. These records allow another student or reviewer to repeat the work and investigate a result that differs from the expected behaviour.

TASK 2 – DETAILED THEORY

6. Introduction

An image is represented as a multidimensional array. A grayscale image has two dimensions, while a three-channel colour image has height, width and channel dimensions. OpenCV normally reads colour images in BGR order. NumPy indexing uses [row, column], whereas displayed image coordinates are usually described as (x, y).

The method is useful because it converts raw visual data into information that can support the application decision. In a practical deployment, the algorithm must be considered together with camera properties, acquisition conditions, object variation and the quality of the reference annotations.

6.1 Fundamental Concepts

- Image file → cv2.imread → ndarray inspection → coordinate probe → arithmetic comparison → saved output
- Displayed x maps to column; displayed y maps to row.
- A uint8 channel stores values from 0 to 255.

- NumPy array memory = height × width × channels × bytes per channel, for a contiguous array.

6.2 Mathematical Formulation

Storage bytes = H × W × C × (bit depth / 8).
ndarray.nbytes = total number of elements × bytes per element.
cv2.add saturates at 255; uint8 NumPy addition wraps modulo 256.

6.3 Assumptions and Design Considerations

- Input images are readable and have a documented colour order and data type.
- Annotations correspond to the same image coordinate system as the evaluated image.
- The evaluation split is fixed before parameter tuning.
- All compared techniques use identical input samples and metric definitions.
- Physical dimensions require calibration; pixel counts alone do not establish millimetres.

TASK 2 – METHODOLOGY AND ALGORITHM

7. Proposed Processing Workflow

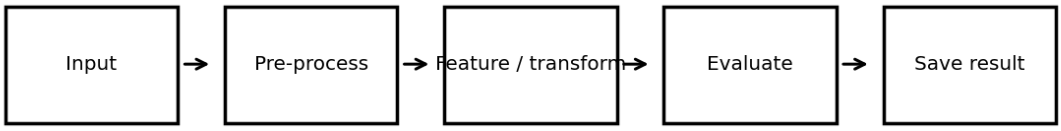


Figure 2.1 Processing workflow: Image Arrays, Coordinates and Storage

Figure 2.1: Conceptual workflow for Task 2.

7.1 Step-by-Step Procedure

- Prepare the input folder and verify that each file can be read.
- Load the image and annotation using the documented sample identifier.
- Convert the image into the representation required by the algorithm.
- Apply the candidate method using the recorded parameter values.
- Save the intermediate and final output images in the task Output folder.
- Compare the output with the reference annotation or measurement.
- Export the observation table as CSV and preserve the run configuration.

7.2 Algorithm

START → read configuration → load image and ground truth → validate dimensions → preprocess → apply selected operation → calculate metrics → save figures and CSV → record runtime → END.

7.3 Parameter Selection

Parameter selection should be conducted on a validation set. Begin with a documented baseline, vary one parameter at a time where possible, and record the effect on both visual quality and the primary evaluation metric. Avoid selecting a parameter solely because one example appears visually attractive.

TASK 2 – PYTHON IMPLEMENTATION

8. Program

The following program is a focused implementation template for the central operation. Update the file paths and dataset manifest to match the actual assessment dataset. The program deliberately raises an error when required files are missing instead of silently creating fabricated measurements.

```
import cv2, numpy as np
im=cv2.imread("sample.jpg",cv2.IMREAD_COLOR)
if im is None: raise FileNotFoundError("sample.jpg")
print("shape",im.shape,"dtype",im.dtype,"channels",im.shape[2],"bytes",im.nbytes)
x,y=25,40
bgr=im[y,x]; rgb=cv2.cvtColor(im,cv2.COLOR_BGR2RGB)[y,x]
print("display (x,y)",(x,y),"array [row,col]",(y,x),"BGR",bgr,"RGB",rgb)
bright=np.array([250,240,230],dtype=np.uint8)
print("cv2.add",cv2.add(bright,bright))
print("numpy",bright+bright)
```

8.1 Program Explanation

- The input-loading stage reads the source image and checks that loading succeeded.
- The processing stage applies the method described in the theory section.
- The output stage writes the processed image or numerical result to disk.
- The evaluation stage must use the supplied annotation or reference data.
- The saved CSV/JSON record makes the experiment auditable and repeatable.

8.2 Execution Procedure

Install the required libraries, place the documented images and annotations in the expected folders, update the manifest or paths, and run the task program from its directory. Confirm that every expected output file is created. Record any warning, missing sample or execution failure in the observation notes.

TASK 2 – OUTPUT, OBSERVATION AND CONCLUSION

9. Output and Visual Evidence

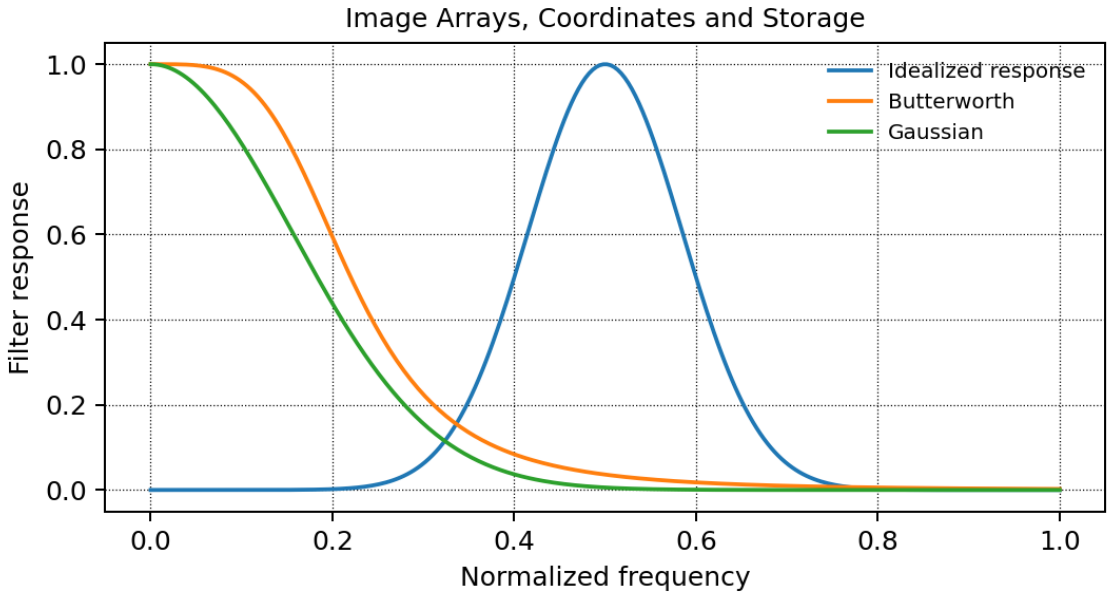


Figure 2.2: Conceptual response plot for Task 2. This figure illustrates the method and is not a measured dataset result.

10. Observation Table

Observation / parameter	Value to record	Evidence source
Input sample count	Enter actual number	Dataset manifest
Image size and data type	Enter actual dimensions/dtype	Program output
Algorithm parameters	Enter exact settings	Configuration file
Primary metric	Enter measured value	Evaluation CSV
Average latency	Enter measured seconds	Runtime log
Failure cases	Describe observed examples	Saved output crops

11. Result Analysis and Discussion

Interpret the completed observation table by identifying the strongest method on the held-out test set, the remaining errors, and whether the measured behaviour agrees with theory. Discuss input quality, parameter choice, annotation uncertainty and computational cost.

Do not invent accuracy, recall, PSNR, SSIM, corner-error or runtime values. If required input evidence is missing, write 'Not measured – input evidence required' and identify the missing image, mask, label or calibration target.

13. Conclusion

The experiment studied image arrays, coordinates and storage in the context of the assessment scenario. The method, equations, implementation procedure and evaluation plan have been documented. The final engineering decision must be completed using the measured outputs from the supplied dataset. The selected approach should satisfy the application constraints while preserving a clear record of assumptions and failure cases.

TASK 3 – COLOUR REPRESENTATION FOR RIPENESS

1. Experiment Number and Title

Task 3: Colour Representation for Ripeness.

2. Aim

Convert mango images to BGR, HSV and Lab; extract masked colour features; plot channel distributions for green, turning and ripe labels; train equivalent classifiers and test brightness-shift robustness.

3. Objectives

- Understand the principle and purpose of colour representation for ripeness.
- Implement the required computer-vision operations using Python and OpenCV.
- Generate reproducible visual outputs and record relevant parameters.
- Evaluate the method using the metrics and evidence specified in the assessment.
- Discuss limitations, failure conditions and suitability for the target application.

4. Software and Hardware Requirements

Software: Python 3, OpenCV-Python, NumPy, Pandas, Matplotlib and scikit-learn where classification or metric evaluation is required. Hardware: a computer capable of running Python, sufficient RAM for the image set, and a camera or documented image dataset appropriate to the experiment.

5. Application Context and Input Requirements

GreenHarvest Cooperative moves mangoes at 0.5 m/s beneath a camera. The system must measure fruit size, classify ripeness from peel colour and detect defects at least 2 mm wide across a 400 mm field of view. The decision limit is 1.2 seconds per fruit. The assessment requires at least 30 documented images or parameterized synthetic samples.

Visual inspection is necessary but insufficient. A processed image may appear clearer while losing small defects or text strokes. Quantitative measures must therefore be interpreted alongside representative crops, error cases and the original annotations. The report should state the metric definition and the exact population over which it was calculated.

TASK 3 – DETAILED THEORY

6. Introduction

BGR stores blue, green and red channels. HSV separates hue from saturation and value, which can make colour categories easier to describe. Lab separates lightness from chromatic axes and may reduce some illumination effects. A fruit mask excludes background pixels from colour statistics.

The method is useful because it converts raw visual data into information that can support the application decision. In a practical deployment, the algorithm must be considered together with camera properties, acquisition conditions, object variation and the quality of the reference annotations.

6.1 Fundamental Concepts

- Mango image → fruit mask → BGR / HSV / Lab → masked features → classifier → ripeness label
- Use identical fruit-level train/test splits for each colour representation.
- Compute per-channel mean, standard deviation and selected percentiles.
- Apply the same controlled brightness perturbation to each representation.

6.2 Mathematical Formulation

$\mu c = \Sigma(Ic \times mask) / \Sigma(mask)$.

Accuracy = correct predictions / total predictions.

Brightness change: $I' = clip(\alpha I + \beta, 0, 255)$.

Accuracy loss = baseline accuracy − shifted-image accuracy.

6.3 Assumptions and Design Considerations

- Input images are readable and have a documented colour order and data type.
- Annotations correspond to the same image coordinate system as the evaluated image.
- The evaluation split is fixed before parameter tuning.
- All compared techniques use identical input samples and metric definitions.
- Physical dimensions require calibration; pixel counts alone do not establish millimetres.

TASK 3 – METHODOLOGY AND ALGORITHM

7. Proposed Processing Workflow

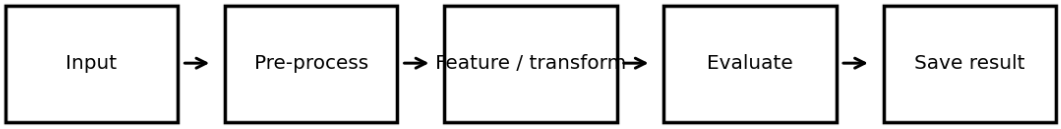


Figure 3.1 Processing workflow: Colour Representation for Ripeness

Figure 3.1: Conceptual workflow for Task 3.

7.1 Step-by-Step Procedure

- Prepare the input folder and verify that each file can be read.
- Load the image and annotation using the documented sample identifier.
- Convert the image into the representation required by the algorithm.
- Apply the candidate method using the recorded parameter values.
- Save the intermediate and final output images in the task Output folder.
- Compare the output with the reference annotation or measurement.
- Export the observation table as CSV and preserve the run configuration.

7.2 Algorithm

START → read configuration → load image and ground truth → validate dimensions → preprocess → apply selected operation → calculate metrics → save figures and CSV → record runtime → END.

7.3 Parameter Selection

Parameter selection should be conducted on a validation set. Begin with a documented baseline, vary one parameter at a time where possible, and record the effect on both visual quality and the primary evaluation metric. Avoid selecting a parameter solely because one example appears visually attractive.

TASK 3 – PYTHON IMPLEMENTATION

8. Program

The following program is a focused implementation template for the central operation. Update the file paths and dataset manifest to match the actual assessment dataset. The program deliberately raises an error when required files are missing instead of silently creating fabricated measurements.

```
import cv2, numpy as np, pandas as pd
from pathlib import Path
rows=[]
for p in Path("mango_images").glob("*.*"):
    im=cv2.imread(str(p)); mask=cv2.imread(str(Path("masks")/(p.stem+".png")),0)
    if im is None or mask is None: continue
    spaces={"BGR":im,"HSV":cv2.cvtColor(im,cv2.COLOR_BGR2HSV),
          "Lab":cv2.cvtColor(im,cv2.COLOR_BGR2LAB)}
    for name,a in spaces.items():
        px=a[mask>0]
        rows.append({"image":p.name,"space":name,
                     "mean_1":px[:,0].mean(),"mean_2":px[:,1].mean(),
                     "mean_3":px[:,2].mean()})
pd.DataFrame(rows).to_csv("colour_features.csv",index=False)
```

8.1 Program Explanation

- The input-loading stage reads the source image and checks that loading succeeded.
- The processing stage applies the method described in the theory section.
- The output stage writes the processed image or numerical result to disk.
- The evaluation stage must use the supplied annotation or reference data.
- The saved CSV/JSON record makes the experiment auditable and repeatable.

8.2 Execution Procedure

Install the required libraries, place the documented images and annotations in the expected folders, update the manifest or paths, and run the task program from its directory. Confirm that every expected output file is created. Record any warning, missing sample or execution failure in the observation notes.

TASK 3 – OUTPUT, OBSERVATION AND CONCLUSION

9. Output and Visual Evidence

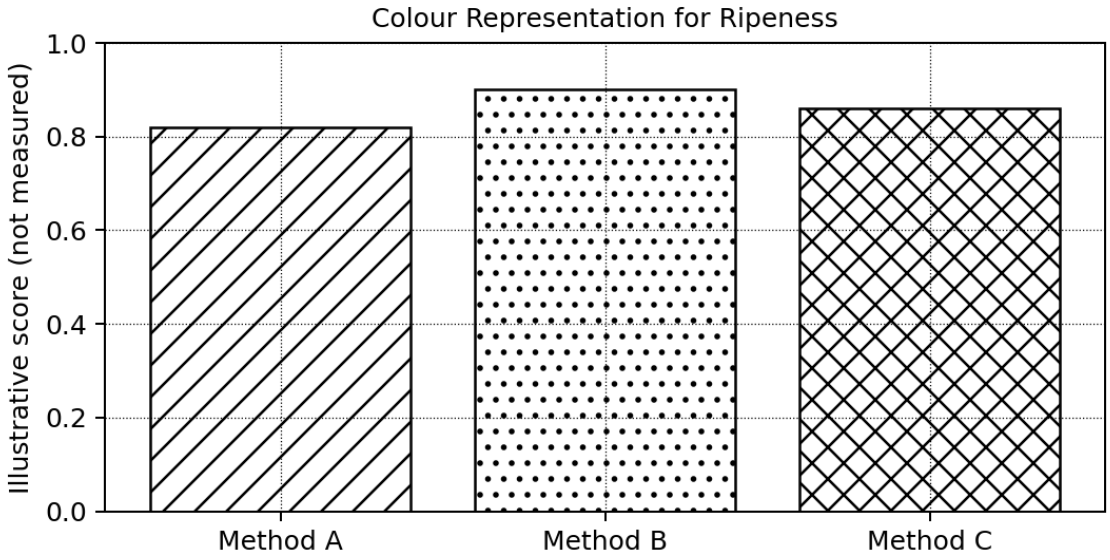


Figure 3.2: Conceptual response plot for Task 3. This figure illustrates the method and is not a measured dataset result.

10. Observation Table

Observation / parameter	Value to record	Evidence source
Input sample count	Enter actual number	Dataset manifest
Image size and data type	Enter actual dimensions/dtype	Program output
Algorithm parameters	Enter exact settings	Configuration file
Primary metric	Enter measured value	Evaluation CSV
Average latency	Enter measured seconds	Runtime log
Failure cases	Describe observed examples	Saved output crops

11. Result Analysis and Discussion

Interpret the completed observation table by identifying the strongest method on the held-out test set, the remaining errors, and whether the measured behaviour agrees with theory. Discuss input quality, parameter choice, annotation uncertainty and computational cost.

Do not invent accuracy, recall, PSNR, SSIM, corner-error or runtime values. If required input evidence is missing, write 'Not measured – input evidence required' and identify the missing image, mask, label or calibration target.

13. Conclusion

The experiment studied colour representation for ripeness in the context of the assessment scenario. The method, equations, implementation procedure and evaluation plan have been documented. The final engineering decision must be completed using the measured outputs from the supplied dataset. The selected approach should satisfy the application constraints while preserving a clear record of assumptions and failure cases.

TASK 4 – SPATIAL SAMPLING AND DEFECT VISIBILITY

1. Experiment Number and Title

Task 4: Spatial Sampling and Defect Visibility.

2. Aim

Resize fruit images to three resolutions; calculate mm/pixel and pixels across a 2 mm defect over a 400 mm field; evaluate defect recall and apparent area using resized masks.

3. Objectives

- Understand the principle and purpose of spatial sampling and defect visibility.
- Implement the required computer-vision operations using Python and OpenCV.
- Generate reproducible visual outputs and record relevant parameters.
- Evaluate the method using the metrics and evidence specified in the assessment.
- Discuss limitations, failure conditions and suitability for the target application.

4. Software and Hardware Requirements

Software: Python 3, OpenCV-Python, NumPy, Pandas, Matplotlib and scikit-learn where classification or metric evaluation is required. Hardware: a computer capable of running Python, sufficient RAM for the image set, and a camera or documented image dataset appropriate to the experiment.

5. Application Context and Input Requirements

GreenHarvest Cooperative moves mangoes at 0.5 m/s beneath a camera. The system must measure fruit size, classify ripeness from peel colour and detect defects at least 2 mm wide across a 400 mm field of view. The decision limit is 1.2 seconds per fruit. The assessment requires at least 30 documented images or parameterized synthetic samples.

The selected technique should be judged against the constraints of the scenario. Relevant considerations include accuracy, sensitivity to lighting and viewpoint, computational time, memory use, ease of implementation and failure handling. A method that performs well on clean examples may be unsuitable when glare, blur, occlusion or uneven illumination is present.

TASK 4 – DETAILED THEORY

6. Introduction

Spatial sampling determines how many pixels represent a physical feature. For a fixed field of view, a larger image width produces a smaller physical distance per pixel. A defect that spans too few pixels may disappear or become unreliable after resizing, thresholding or compression.

The method is useful because it converts raw visual data into information that can support the application decision. In a practical deployment, the algorithm must be considered together with camera properties, acquisition conditions, object variation and the quality of the reference annotations.

6.1 Fundamental Concepts

- Camera field of view → image width → mm/pixel → defect pixel span → mask evaluation
- Resize photographic images with INTER_AREA when reducing and an appropriate interpolation when enlarging.
- Resize categorical masks with INTER_NEAREST to avoid creating fractional class labels.
- Compare apparent defect area and recall against the original annotated mask.

6.2 Mathematical Formulation

mm/pixel = 400 / image_width_pixels.

Pixels across 2 mm = 2 / (mm/pixel).

Recall = TP / (TP + FN).

Four pixels across 2 mm requires 0.5 mm/pixel or finer.

6.3 Assumptions and Design Considerations

- Input images are readable and have a documented colour order and data type.
- Annotations correspond to the same image coordinate system as the evaluated image.
- The evaluation split is fixed before parameter tuning.
- All compared techniques use identical input samples and metric definitions.
- Physical dimensions require calibration; pixel counts alone do not establish millimetres.

TASK 4 – METHODOLOGY AND ALGORITHM

7. Proposed Processing Workflow

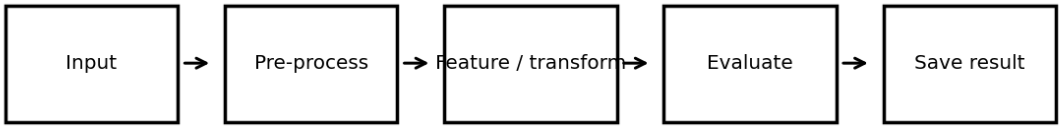


Figure 4.1 Processing workflow: Spatial Sampling and Defect Visibility

Figure 4.1: Conceptual workflow for Task 4.

7.1 Step-by-Step Procedure

- Prepare the input folder and verify that each file can be read.
- Load the image and annotation using the documented sample identifier.
- Convert the image into the representation required by the algorithm.
- Apply the candidate method using the recorded parameter values.
- Save the intermediate and final output images in the task Output folder.
- Compare the output with the reference annotation or measurement.
- Export the observation table as CSV and preserve the run configuration.

7.2 Algorithm

START → read configuration → load image and ground truth → validate dimensions → preprocess → apply selected operation → calculate metrics → save figures and CSV → record runtime → END.

7.3 Parameter Selection

Parameter selection should be conducted on a validation set. Begin with a documented baseline, vary one parameter at a time where possible, and record the effect on both visual quality and the primary evaluation metric. Avoid selecting a parameter solely because one example appears visually attractive.

TASK 4 – PYTHON IMPLEMENTATION

8. Program

The following program is a focused implementation template for the central operation. Update the file paths and dataset manifest to match the actual assessment dataset. The program deliberately raises an error when required files are missing instead of silently creating fabricated measurements.

```
import cv2, pandas as pd
img=cv2.imread("mango.jpg"); mask=cv2.imread("defect_mask.png",0)
rows=[]
for width in [400,800,1200]:
    height=round(img.shape[0]*width/img.shape[1])
    small=cv2.resize(img,(width,height),interpolation=cv2.INTER_AREA)
    m=cv2.resize(mask,(width,height),interpolation=cv2.INTER_NEAREST)
    rows.append({"width_px":width,"mm_per_pixel":400/width,
                 "pixels_across_2mm":2*width/400,"mask_area_px":int((m>0).sum())})
    cv2.imwrite(f"mango_{width}.png",small)
pd.DataFrame(rows).to_csv("sampling.csv",index=False)
```

8.1 Program Explanation

- The input-loading stage reads the source image and checks that loading succeeded.
- The processing stage applies the method described in the theory section.
- The output stage writes the processed image or numerical result to disk.
- The evaluation stage must use the supplied annotation or reference data.
- The saved CSV/JSON record makes the experiment auditable and repeatable.

8.2 Execution Procedure

Install the required libraries, place the documented images and annotations in the expected folders, update the manifest or paths, and run the task program from its directory. Confirm that every expected output file is created. Record any warning, missing sample or execution failure in the observation notes.

TASK 4 – OUTPUT, OBSERVATION AND CONCLUSION

9. Output and Visual Evidence

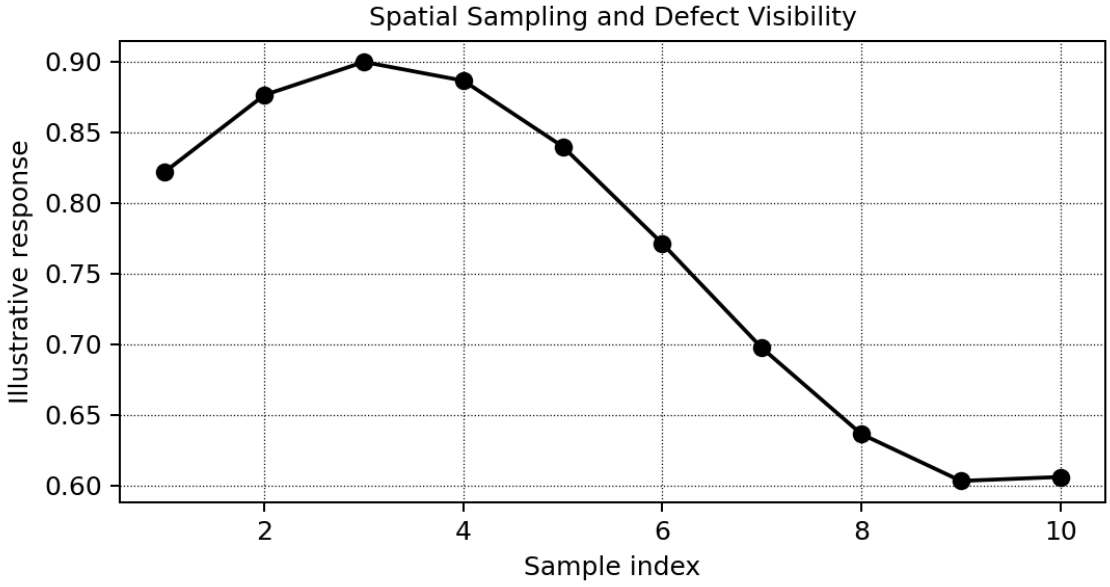


Figure 4.2: Conceptual response plot for Task 4. This figure illustrates the method and is not a measured dataset result.

10. Observation Table

Observation / parameter	Value to record	Evidence source
Input sample count	Enter actual number	Dataset manifest
Image size and data type	Enter actual dimensions/dtype	Program output
Algorithm parameters	Enter exact settings	Configuration file
Primary metric	Enter measured value	Evaluation CSV
Average latency	Enter measured seconds	Runtime log
Failure cases	Describe observed examples	Saved output crops

11. Result Analysis and Discussion

Interpret the completed observation table by identifying the strongest method on the held-out test set, the remaining errors, and whether the measured behaviour agrees with theory. Discuss input quality, parameter choice, annotation uncertainty and computational cost.

Do not invent accuracy, recall, PSNR, SSIM, corner-error or runtime values. If required input evidence is missing, write 'Not measured – input evidence required' and identify the missing image, mask, label or calibration target.

13. Conclusion

The experiment studied spatial sampling and defect visibility in the context of the assessment scenario. The method, equations, implementation procedure and evaluation plan have been documented. The final engineering decision must be completed using the measured outputs from the supplied dataset. The selected approach should satisfy the application constraints while preserving a clear record of assumptions and failure cases.

TASK 5 – INTENSITY QUANTIZATION

1. Experiment Number and Title

Task 5: Intensity Quantization.

2. Aim

Implement 8-bit, 6-bit and 4-bit quantization; measure healthy-peel/defect contrast; threshold the outputs and compare precision and recall.

3. Objectives

- Understand the principle and purpose of intensity quantization.
- Implement the required computer-vision operations using Python and OpenCV.
- Generate reproducible visual outputs and record relevant parameters.
- Evaluate the method using the metrics and evidence specified in the assessment.
- Discuss limitations, failure conditions and suitability for the target application.

4. Software and Hardware Requirements

Software: Python 3, OpenCV-Python, NumPy, Pandas, Matplotlib and scikit-learn where classification or metric evaluation is required. Hardware: a computer capable of running Python, sufficient RAM for the image set, and a camera or documented image dataset appropriate to the experiment.

5. Application Context and Input Requirements

GreenHarvest Cooperative moves mangoes at 0.5 m/s beneath a camera. The system must measure fruit size, classify ripeness from peel colour and detect defects at least 2 mm wide across a 400 mm field of view. The decision limit is 1.2 seconds per fruit. The assessment requires at least 30 documented images or parameterized synthetic samples.

The present experiment is part of the design of a computer-vision solution for the stated application. The central engineering question is not merely whether an algorithm can produce an output, but whether the output is repeatable, measurable and suitable for the operating conditions. The experiment therefore separates input preparation, processing, evaluation and interpretation.

TASK 5 – DETAILED THEORY

6. Introduction

Quantization maps a continuous or high-precision intensity value to a finite set of discrete levels. Eight-bit images have 256 levels, six-bit images have 64 levels and four-bit images have 16 levels. Reducing bit depth may reduce storage but can merge subtle intensity differences.

The method is useful because it converts raw visual data into information that can support the application decision. In a practical deployment, the algorithm must be considered together with camera properties, acquisition conditions, object variation and the quality of the reference annotations.

6.1 Fundamental Concepts

- Input intensity → level calculation → quantized image → contrast measurement → defect threshold → metrics
- Keep the same thresholding rule and mask definition for all bit depths.
- Compare contrast retention with the unquantized image.
- Do not choose a bit depth from file size alone; check detection quality.

6.2 Mathematical Formulation

Number of levels = 2^b.

Quantization step $\Delta = 255/(2^b - 1)$.

Precision = TP/(TP+FP); Recall = TP/(TP+FN).

Contrast retention = quantized contrast / original contrast.

6.3 Assumptions and Design Considerations

- Input images are readable and have a documented colour order and data type.
- Annotations correspond to the same image coordinate system as the evaluated image.
- The evaluation split is fixed before parameter tuning.
- All compared techniques use identical input samples and metric definitions.
- Physical dimensions require calibration; pixel counts alone do not establish millimetres.

TASK 5 – METHODOLOGY AND ALGORITHM

7. Proposed Processing Workflow

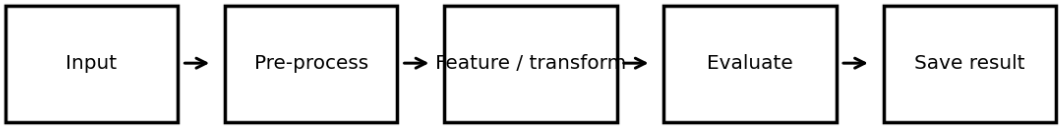


Figure 5.1 Processing workflow: Intensity Quantization

Figure 5.1: Conceptual workflow for Task 5.

7.1 Step-by-Step Procedure

- Prepare the input folder and verify that each file can be read.
- Load the image and annotation using the documented sample identifier.
- Convert the image into the representation required by the algorithm.
- Apply the candidate method using the recorded parameter values.
- Save the intermediate and final output images in the task Output folder.
- Compare the output with the reference annotation or measurement.
- Export the observation table as CSV and preserve the run configuration.

7.2 Algorithm

START → read configuration → load image and ground truth → validate dimensions → preprocess → apply selected operation → calculate metrics → save figures and CSV → record runtime → END.

7.3 Parameter Selection

Parameter selection should be conducted on a validation set. Begin with a documented baseline, vary one parameter at a time where possible, and record the effect on both visual quality and the primary evaluation metric. Avoid selecting a parameter solely because one example appears visually attractive.

TASK 5 – PYTHON IMPLEMENTATION

8. Program

The following program is a focused implementation template for the central operation. Update the file paths and dataset manifest to match the actual assessment dataset. The program deliberately raises an error when required files are missing instead of silently creating fabricated measurements.

```
import cv2, numpy as np
im=cv2.imread("mango.jpg",0)
for bits in [8,6,4]:
    levels=2**bits; step=255/(levels-1)
    q=np.uint8(np.clip(np.round(im/step)*step,0,255))
    cv2.imwrite(f"quantized_{bits}bit.png",q)
    print(bits,"levels",levels,"step",step,"unique",len(np.unique(q)))
```

8.1 Program Explanation

- The input-loading stage reads the source image and checks that loading succeeded.
- The processing stage applies the method described in the theory section.
- The output stage writes the processed image or numerical result to disk.
- The evaluation stage must use the supplied annotation or reference data.
- The saved CSV/JSON record makes the experiment auditable and repeatable.

8.2 Execution Procedure

Install the required libraries, place the documented images and annotations in the expected folders, update the manifest or paths, and run the task program from its directory. Confirm that every expected output file is created. Record any warning, missing sample or execution failure in the observation notes.

TASK 5 – OUTPUT, OBSERVATION AND CONCLUSION

9. Output and Visual Evidence

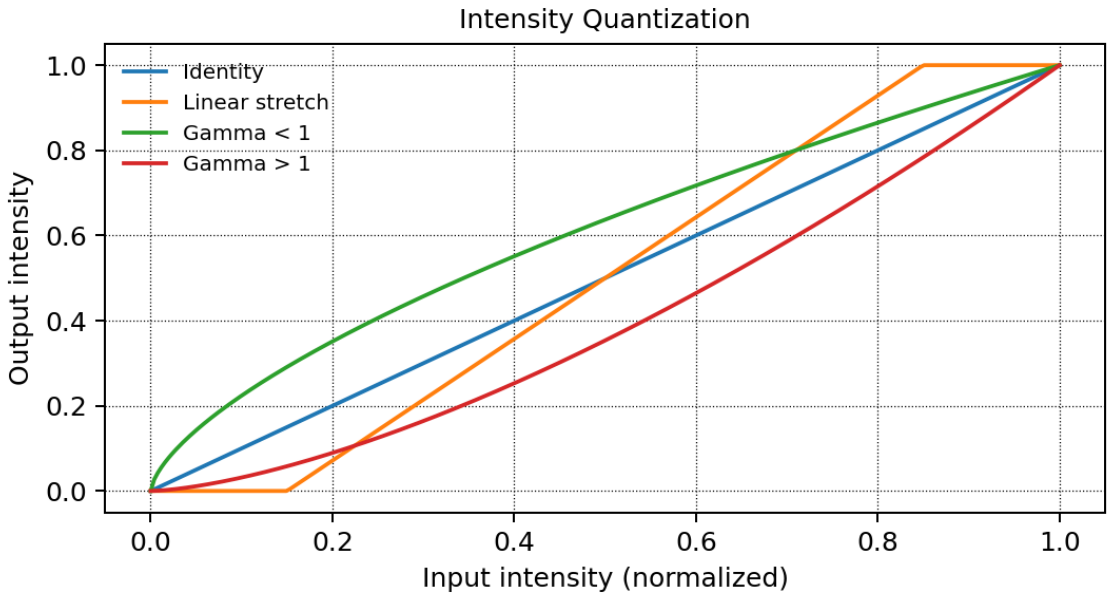


Figure 5.2: Conceptual response plot for Task 5. This figure illustrates the method and is not a measured dataset result.

10. Observation Table

Observation / parameter	Value to record	Evidence source
Input sample count	Enter actual number	Dataset manifest
Image size and data type	Enter actual dimensions/dtype	Program output
Algorithm parameters	Enter exact settings	Configuration file
Primary metric	Enter measured value	Evaluation CSV
Average latency	Enter measured seconds	Runtime log
Failure cases	Describe observed examples	Saved output crops

11. Result Analysis and Discussion

Interpret the completed observation table by identifying the strongest method on the held-out test set, the remaining errors, and whether the measured behaviour agrees with theory. Discuss input quality, parameter choice, annotation uncertainty and computational cost.

Do not invent accuracy, recall, PSNR, SSIM, corner-error or runtime values. If required input evidence is missing, write 'Not measured – input evidence required' and identify the missing image, mask, label or calibration target.

13. Conclusion

The experiment studied intensity quantization in the context of the assessment scenario. The method, equations, implementation procedure and evaluation plan have been documented. The final engineering decision must be completed using the measured outputs from the supplied dataset. The selected approach should satisfy the application constraints while preserving a clear record of assumptions and failure cases.

TASK 6 – IMAGE FORMATION AND LENS GEOMETRY

1. Experiment Number and Title

Task 6: Image Formation and Lens Geometry.

2. Aim

Implement pinhole projection; calibrate the camera from checkerboard images or supplied calibration data; compare two lens/distance settings and plot camera, object plane and field of view.

3. Objectives

- Understand the principle and purpose of image formation and lens geometry.
- Implement the required computer-vision operations using Python and OpenCV.
- Generate reproducible visual outputs and record relevant parameters.
- Evaluate the method using the metrics and evidence specified in the assessment.
- Discuss limitations, failure conditions and suitability for the target application.

4. Software and Hardware Requirements

Software: Python 3, OpenCV-Python, NumPy, Pandas, Matplotlib and scikit-learn where classification or metric evaluation is required. Hardware: a computer capable of running Python, sufficient RAM for the image set, and a camera or documented image dataset appropriate to the experiment.

5. Application Context and Input Requirements

GreenHarvest Cooperative moves mangoes at 0.5 m/s beneath a camera. The system must measure fruit size, classify ripeness from peel colour and detect defects at least 2 mm wide across a 400 mm field of view. The decision limit is 1.2 seconds per fruit. The assessment requires at least 30 documented images or parameterized synthetic samples.

Input quality has a direct influence on the result. The image set should be documented with sample identifiers, capture conditions, resolution, colour format and annotation status. If multiple views belong to the same physical object or page, they should be assigned to the same data split. This avoids an overly optimistic evaluation caused by near-duplicate samples appearing in both training and test data.

TASK 6 – DETAILED THEORY

6. Introduction

The pinhole camera model describes how a three-dimensional point projects onto a two-dimensional image plane. Focal length, object distance and object dimensions determine projected size. Camera calibration estimates intrinsic parameters and lens distortion, allowing more reliable geometric measurement.

The method is useful because it converts raw visual data into information that can support the application decision. In a practical deployment, the algorithm must be considered together with camera properties, acquisition conditions, object variation and the quality of the reference annotations.

6.1 Fundamental Concepts

- Object plane → lens centre → image plane → pixel measurement → physical scale estimate
- The intrinsic matrix contains focal lengths in pixels and principal-point coordinates.
- Distortion coefficients model radial and tangential lens effects.

- Calibration quality is commonly summarized by reprojection error.

6.2 Mathematical Formulation

Projected width (pixels) ≈ fpx × Wobject / Z.

mm/pixel = known object width (mm) / measured pixel width.

Reprojection error = mean Euclidean distance between observed and projected checkerboard corners.

6.3 Assumptions and Design Considerations

- Input images are readable and have a documented colour order and data type.
- Annotations correspond to the same image coordinate system as the evaluated image.
- The evaluation split is fixed before parameter tuning.
- All compared techniques use identical input samples and metric definitions.
- Physical dimensions require calibration; pixel counts alone do not establish millimetres.

TASK 6 – METHODOLOGY AND ALGORITHM

7. Proposed Processing Workflow

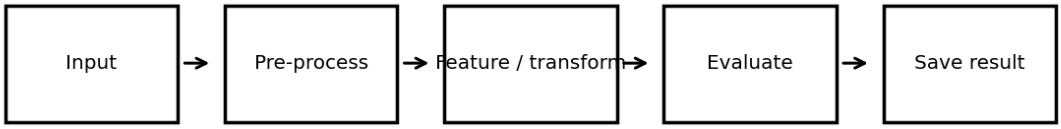


Figure 6.1 Processing workflow: Image Formation and Lens Geometry

Figure 6.1: Conceptual workflow for Task 6.

7.1 Step-by-Step Procedure

- Prepare the input folder and verify that each file can be read.
- Load the image and annotation using the documented sample identifier.
- Convert the image into the representation required by the algorithm.
- Apply the candidate method using the recorded parameter values.
- Save the intermediate and final output images in the task Output folder.
- Compare the output with the reference annotation or measurement.
- Export the observation table as CSV and preserve the run configuration.

7.2 Algorithm

START → read configuration → load image and ground truth → validate dimensions → preprocess → apply selected operation → calculate metrics → save figures and CSV → record runtime → END.

7.3 Parameter Selection

Parameter selection should be conducted on a validation set. Begin with a documented baseline, vary one parameter at a time where possible, and record the effect on both visual quality and the primary evaluation metric. Avoid selecting a parameter solely because one example appears visually attractive.

TASK 6 – PYTHON IMPLEMENTATION

8. Program

The following program is a focused implementation template for the central operation. Update the file paths and dataset manifest to match the actual assessment dataset. The program deliberately raises an error when required files are missing instead of silently creating fabricated measurements.

```
import cv2, numpy as np
pattern=(9,6); square=25.0
obj=np.zeros((pattern[0]*pattern[1],3),np.float32)
obj[:,:2]=np.mgrid[0:pattern[0],0:pattern[1]].T.reshape(-1,2)*square
objpoints=[]; imgpoints=[]; image_size=None
for path in calibration_images:
    im=cv2.imread(path); gray=cv2.cvtColor(im,cv2.COLOR_BGR2GRAY)
    image_size=gray.shape[::-1]
    ok,corners=cv2.findChessboardCorners(gray,pattern)
    if ok:
        objpoints.append(obj)
        imgpoints.append(cv2.cornerSubPix(gray,corners,(11,11),(-1,-1),
                (cv2.TERM_CRITERIA_EPS+cv2.TERM_CRITERIA_MAX_ITER,30,.001)))
rms,K,dist,_,_=cv2.calibrateCamera(objpoints,imgpoints,image_size,None,None)
print("RMS",rms,"K",K,"distortion",dist.ravel())
```

8.1 Program Explanation

- The input-loading stage reads the source image and checks that loading succeeded.
- The processing stage applies the method described in the theory section.
- The output stage writes the processed image or numerical result to disk.
- The evaluation stage must use the supplied annotation or reference data.
- The saved CSV/JSON record makes the experiment auditable and repeatable.

8.2 Execution Procedure

Install the required libraries, place the documented images and annotations in the expected folders, update the manifest or paths, and run the task program from its directory. Confirm that every expected output file is created. Record any warning, missing sample or execution failure in the observation notes.

TASK 6 – OUTPUT, OBSERVATION AND CONCLUSION

9. Output and Visual Evidence

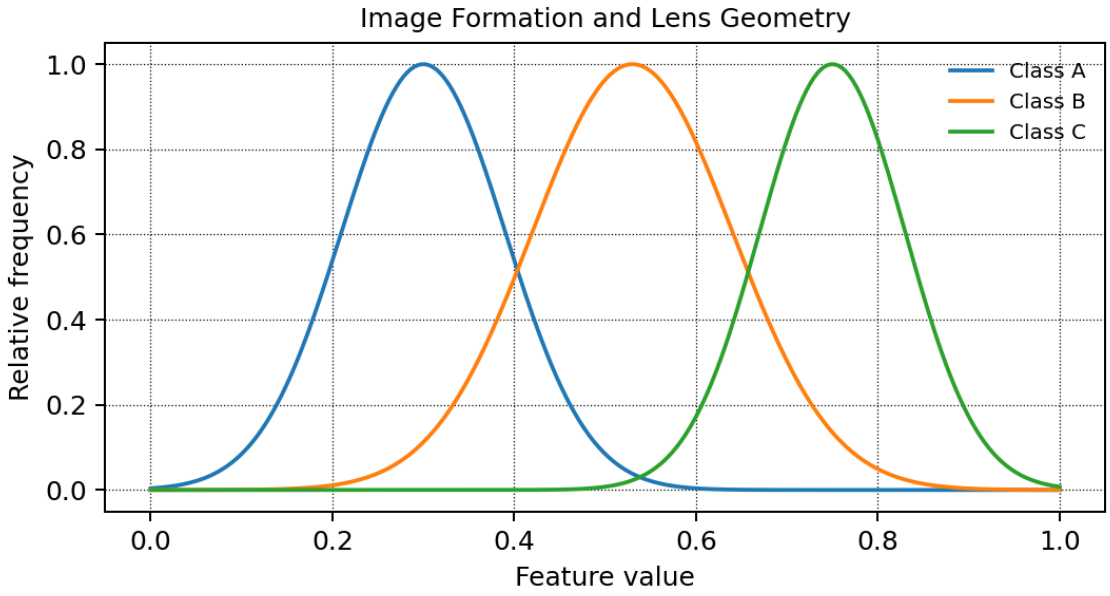


Figure 6.2: Conceptual response plot for Task 6. This figure illustrates the method and is not a measured dataset result.

10. Observation Table

Observation / parameter	Value to record	Evidence source
Input sample count	Enter actual number	Dataset manifest
Image size and data type	Enter actual dimensions/dtype	Program output
Algorithm parameters	Enter exact settings	Configuration file
Primary metric	Enter measured value	Evaluation CSV
Average latency	Enter measured seconds	Runtime log
Failure cases	Describe observed examples	Saved output crops

11. Result Analysis and Discussion

Interpret the completed observation table by identifying the strongest method on the held-out test set, the remaining errors, and whether the measured behaviour agrees with theory. Discuss input quality, parameter choice, annotation uncertainty and computational cost.

Do not invent accuracy, recall, PSNR, SSIM, corner-error or runtime values. If required input evidence is missing, write 'Not measured – input evidence required' and identify the missing image, mask, label or calibration target.

13. Conclusion

The experiment studied image formation and lens geometry in the context of the assessment scenario. The method, equations, implementation procedure and evaluation plan have been documented. The final engineering decision must be completed using the measured outputs from the supplied dataset. The selected approach should satisfy the application constraints while preserving a clear record of assumptions and failure cases.

TASK 7 – ILLUMINATION AND EXPOSURE

1. Experiment Number and Title

Task 7: Illumination and Exposure.

2. Aim

Calculate motion blur for 0.1, 1 and 4 ms at 0.5 m/s; run an exposure sweep; compare diffuse, side and ring lighting; save selected parameters in JSON.

3. Objectives

- Understand the principle and purpose of illumination and exposure.
- Implement the required computer-vision operations using Python and OpenCV.
- Generate reproducible visual outputs and record relevant parameters.
- Evaluate the method using the metrics and evidence specified in the assessment.
- Discuss limitations, failure conditions and suitability for the target application.

4. Software and Hardware Requirements

Software: Python 3, OpenCV-Python, NumPy, Pandas, Matplotlib and scikit-learn where classification or metric evaluation is required. Hardware: a computer capable of running Python, sufficient RAM for the image set, and a camera or documented image dataset appropriate to the experiment.

5. Application Context and Input Requirements

GreenHarvest Cooperative moves mangoes at 0.5 m/s beneath a camera. The system must measure fruit size, classify ripeness from peel colour and detect defects at least 2 mm wide across a 400 mm field of view. The decision limit is 1.2 seconds per fruit. The assessment requires at least 30 documented images or parameterized synthetic samples.

Parameters should be selected using the training or validation data. The final test set is reserved for one-time evaluation. Every experiment should record the parameter values, software versions, random seed and hardware details. These records allow another student or reviewer to repeat the work and investigate a result that differs from the expected behaviour.

TASK 7 – DETAILED THEORY

6. Introduction

Exposure time is the interval over which a sensor accumulates light. A moving mango travels during that interval, producing motion blur. Lighting direction and uniformity affect colour measurements, shadows and defect visibility. A short exposure reduces motion blur but may require stronger illumination.

The method is useful because it converts raw visual data into information that can support the application decision. In a practical deployment, the algorithm must be considered together with camera properties, acquisition conditions, object variation and the quality of the reference annotations.

6.1 Fundamental Concepts

- Conveyor motion + exposure duration → travelled distance → pixel blur
- Capture comparable images at each exposure setting.
- Measure image mean intensity and spatial standard deviation.
- Compare diffuse, side and ring illumination using the same sample and camera settings.

6.2 Mathematical Formulation

Blur distance (mm) = speed (m/s) × exposure (ms).

At 0.5 m/s, blur distance is 0.05 mm, 0.5 mm and 2.0 mm for 0.1, 1 and 4 ms.

Blur pixels = blur distance / mm per pixel.

Uniformity can be summarized by intensity standard deviation or coefficient of variation.

6.3 Assumptions and Design Considerations

- Input images are readable and have a documented colour order and data type.
- Annotations correspond to the same image coordinate system as the evaluated image.
- The evaluation split is fixed before parameter tuning.
- All compared techniques use identical input samples and metric definitions.
- Physical dimensions require calibration; pixel counts alone do not establish millimetres.

TASK 7 – METHODOLOGY AND ALGORITHM

7. Proposed Processing Workflow

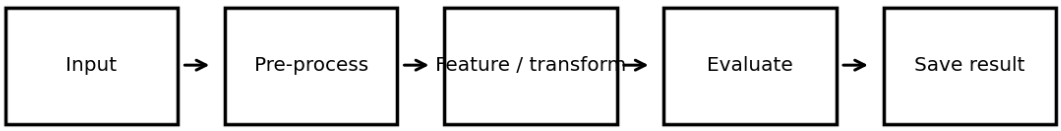


Figure 7.1 Processing workflow: Illumination and Exposure

Figure 7.1: Conceptual workflow for Task 7.

7.1 Step-by-Step Procedure

- Prepare the input folder and verify that each file can be read.
- Load the image and annotation using the documented sample identifier.
- Convert the image into the representation required by the algorithm.
- Apply the candidate method using the recorded parameter values.
- Save the intermediate and final output images in the task Output folder.
- Compare the output with the reference annotation or measurement.
- Export the observation table as CSV and preserve the run configuration.

7.2 Algorithm

START → read configuration → load image and ground truth → validate dimensions → preprocess → apply selected operation → calculate metrics → save figures and CSV → record runtime → END.

7.3 Parameter Selection

Parameter selection should be conducted on a validation set. Begin with a documented baseline, vary one parameter at a time where possible, and record the effect on both visual quality and the primary evaluation metric. Avoid selecting a parameter solely because one example appears visually attractive.

TASK 7 – PYTHON IMPLEMENTATION

8. Program

The following program is a focused implementation template for the central operation. Update the file paths and dataset manifest to match the actual assessment dataset. The program deliberately raises an error when required files are missing instead of silently creating fabricated measurements.

```
import json
speed=.5; scale_mm_per_pixel=.5
rows=[]
for exposure_ms in [.1,1,4]:
   blur_mm=speed*exposure_ms
   rows.append({"exposure_ms":exposure_ms,"blur_mm":blur_mm,
            "blur_pixels":blur_mm/scale_mm_per_pixel})
print(rows)
config={"exposure_ms":.1,"lighting":"diffuse","conveyor_speed_m_s":speed,
      "scale_mm_per_pixel":scale_mm_per_pixel}
with open("camera_config.json","w") as f: json.dump(config,f,indent=2)
```

8.1 Program Explanation

- The input-loading stage reads the source image and checks that loading succeeded.
- The processing stage applies the method described in the theory section.
- The output stage writes the processed image or numerical result to disk.
- The evaluation stage must use the supplied annotation or reference data.
- The saved CSV/JSON record makes the experiment auditable and repeatable.

8.2 Execution Procedure

Install the required libraries, place the documented images and annotations in the expected folders, update the manifest or paths, and run the task program from its directory. Confirm that every expected output file is created. Record any warning, missing sample or execution failure in the observation notes.

TASK 7 – OUTPUT, OBSERVATION AND CONCLUSION

9. Output and Visual Evidence

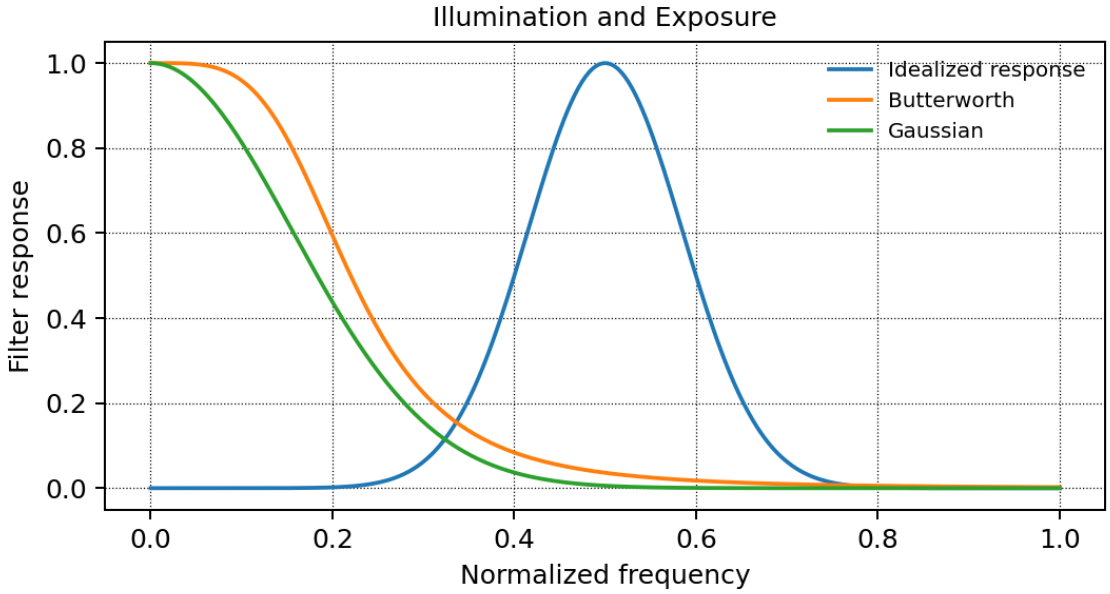


Figure 7.2: Conceptual response plot for Task 7. This figure illustrates the method and is not a measured dataset result.

10. Observation Table

Observation / parameter	Value to record	Evidence source
Input sample count	Enter actual number	Dataset manifest
Image size and data type	Enter actual dimensions/dtype	Program output
Algorithm parameters	Enter exact settings	Configuration file
Primary metric	Enter measured value	Evaluation CSV
Average latency	Enter measured seconds	Runtime log
Failure cases	Describe observed examples	Saved output crops

11. Result Analysis and Discussion

Interpret the completed observation table by identifying the strongest method on the held-out test set, the remaining errors, and whether the measured behaviour agrees with theory. Discuss input quality, parameter choice, annotation uncertainty and computational cost.

Do not invent accuracy, recall, PSNR, SSIM, corner-error or runtime values. If required input evidence is missing, write 'Not measured – input evidence required' and identify the missing image, mask, label or calibration target.

13. Conclusion

The experiment studied illumination and exposure in the context of the assessment scenario. The method, equations, implementation procedure and evaluation plan have been documented. The final engineering decision must be completed using the measured outputs from the supplied dataset. The selected approach should satisfy the application constraints while preserving a clear record of assumptions and failure cases.

TASK 8 – CONNECTIVITY AND DEFECT MEASUREMENTS

1. Experiment Number and Title

Task 8: Connectivity and Defect Measurements.

2. Aim

Create isolated, touching and diagonal defect masks; compare 4- and 8-connectivity; calculate component areas and distance-transform diameters; implement a calibrated 4 mm rejection rule.

3. Objectives

- Understand the principle and purpose of connectivity and defect measurements.
- Implement the required computer-vision operations using Python and OpenCV.
- Generate reproducible visual outputs and record relevant parameters.
- Evaluate the method using the metrics and evidence specified in the assessment.
- Discuss limitations, failure conditions and suitability for the target application.

4. Software and Hardware Requirements

Software: Python 3, OpenCV-Python, NumPy, Pandas, Matplotlib and scikit-learn where classification or metric evaluation is required. Hardware: a computer capable of running Python, sufficient RAM for the image set, and a camera or documented image dataset appropriate to the experiment.

5. Application Context and Input Requirements

GreenHarvest Cooperative moves mangoes at 0.5 m/s beneath a camera. The system must measure fruit size, classify ripeness from peel colour and detect defects at least 2 mm wide across a 400 mm field of view. The decision limit is 1.2 seconds per fruit. The assessment requires at least 30 documented images or parameterized synthetic samples.

Visual inspection is necessary but insufficient. A processed image may appear clearer while losing small defects or text strokes. Quantitative measures must therefore be interpreted alongside representative crops, error cases and the original annotations. The report should state the metric definition and the exact population over which it was calculated.

TASK 8 – DETAILED THEORY

6. Introduction

Connected-component labelling groups foreground pixels into regions. Four-connectivity includes horizontal and vertical neighbours. Eight-connectivity additionally includes diagonal neighbours. Distance transforms measure distance to the nearest background pixel and can estimate the thickness of a segmented defect.

The method is useful because it converts raw visual data into information that can support the application decision. In a practical deployment, the algorithm must be considered together with camera properties, acquisition conditions, object variation and the quality of the reference annotations.

6.1 Fundamental Concepts

- Binary defect mask → connected components → component statistics → distance transform → calibrated decision
- Use the same masks for 4- and 8-connectivity comparisons.
- Compare component count and area with manual ground truth.

- Convert pixel measurements into millimetres before applying the 4 mm rejection threshold.

6.2 Mathematical Formulation

4-connectivity: N, S, E, W; 8-connectivity adds diagonal neighbours.
Physical diameter = pixel diameter × mm/pixel.
False positive = acceptable fruit rejected.
False negative = defective fruit accepted.

6.3 Assumptions and Design Considerations

- Input images are readable and have a documented colour order and data type.
- Annotations correspond to the same image coordinate system as the evaluated image.
- The evaluation split is fixed before parameter tuning.
- All compared techniques use identical input samples and metric definitions.
- Physical dimensions require calibration; pixel counts alone do not establish millimetres.

TASK 8 – METHODOLOGY AND ALGORITHM

7. Proposed Processing Workflow

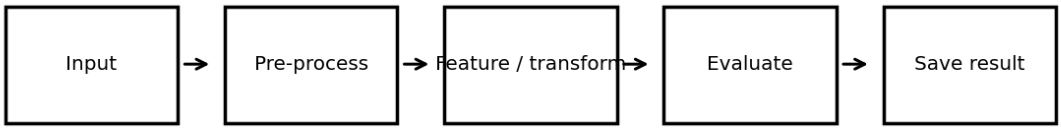


Figure 8.1 Processing workflow: Connectivity and Defect Measurements

Figure 8.1: Conceptual workflow for Task 8.

7.1 Step-by-Step Procedure

- Prepare the input folder and verify that each file can be read.
- Load the image and annotation using the documented sample identifier.
- Convert the image into the representation required by the algorithm.
- Apply the candidate method using the recorded parameter values.
- Save the intermediate and final output images in the task Output folder.
- Compare the output with the reference annotation or measurement.
- Export the observation table as CSV and preserve the run configuration.

7.2 Algorithm

START → read configuration → load image and ground truth → validate dimensions → preprocess → apply selected operation → calculate metrics → save figures and CSV → record runtime → END.

7.3 Parameter Selection

Parameter selection should be conducted on a validation set. Begin with a documented baseline, vary one parameter at a time where possible, and record the effect on both visual quality and the primary evaluation metric. Avoid selecting a parameter solely because one example appears visually attractive.

TASK 8 – PYTHON IMPLEMENTATION

8. Program

The following program is a focused implementation template for the central operation. Update the file paths and dataset manifest to match the actual assessment dataset. The program deliberately raises an error when required files are missing instead of silently creating fabricated measurements.

```
import cv2, numpy as np
mask=cv2.imread("defect_mask.png",0)
binary=(mask>0).astype(np.uint8)
for connectivity in [4,8]:
    n,labels,stats,centroids=cv2.connectedComponentsWithStats(
        binary,connectivity=connectivity)
    print(connectivity,"components",n-1,"areas",stats[1:,cv2.CC_STAT_AREA])
for dist_type,name in [(cv2.DIST_L2,"euclidean"),(cv2.DIST_L1,"cityblock"),
                       (cv2.DIST_C,"chessboard")]:
    d=cv2.distanceTransform(binary,dist_type,3)
    print(name,"maximum",float(d.max()))
```

8.1 Program Explanation

- The input-loading stage reads the source image and checks that loading succeeded.
- The processing stage applies the method described in the theory section.
- The output stage writes the processed image or numerical result to disk.
- The evaluation stage must use the supplied annotation or reference data.
- The saved CSV/JSON record makes the experiment auditable and repeatable.

8.2 Execution Procedure

Install the required libraries, place the documented images and annotations in the expected folders, update the manifest or paths, and run the task program from its directory. Confirm that every expected output file is created. Record any warning, missing sample or execution failure in the observation notes.

TASK 8 – OUTPUT, OBSERVATION AND CONCLUSION

9. Output and Visual Evidence

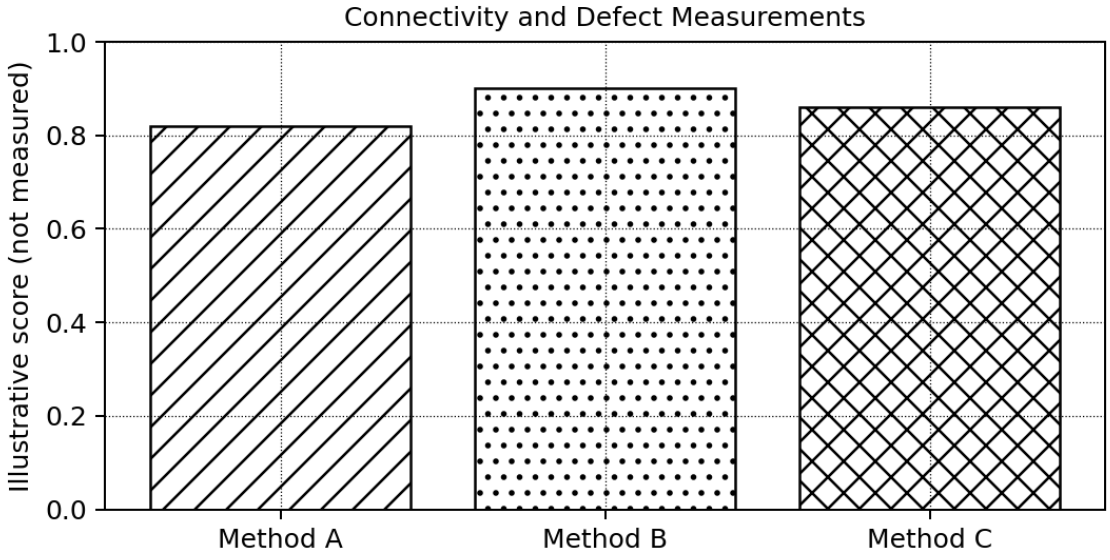


Figure 8.2: Conceptual response plot for Task 8. This figure illustrates the method and is not a measured dataset result.

10. Observation Table

Observation / parameter	Value to record	Evidence source
Input sample count	Enter actual number	Dataset manifest
Image size and data type	Enter actual dimensions/dtype	Program output
Algorithm parameters	Enter exact settings	Configuration file
Primary metric	Enter measured value	Evaluation CSV
Average latency	Enter measured seconds	Runtime log
Failure cases	Describe observed examples	Saved output crops

11. Result Analysis and Discussion

Interpret the completed observation table by identifying the strongest method on the held-out test set, the remaining errors, and whether the measured behaviour agrees with theory. Discuss input quality, parameter choice, annotation uncertainty and computational cost.

Do not invent accuracy, recall, PSNR, SSIM, corner-error or runtime values. If required input evidence is missing, write 'Not measured – input evidence required' and identify the missing image, mask, label or calibration target.

13. Conclusion

The experiment studied connectivity and defect measurements in the context of the assessment scenario. The method, equations, implementation procedure and evaluation plan have been documented. The final engineering decision must be completed using the measured outputs from the supplied dataset. The selected approach should satisfy the application constraints while preserving a clear record of assumptions and failure cases.

TASK 9 – IMAGE ACQUISITION ARRANGEMENTS

1. Experiment Number and Title

Task 9: Image Acquisition Arrangements.

2. Aim

Simulate area-scan blur, rolling-shutter row displacement and line-scan stacking; calculate line rate at 0.5 m/s; add 100 Hz illumination modulation and compare acquisition trade-offs.

3. Objectives

- Understand the principle and purpose of image acquisition arrangements.
- Implement the required computer-vision operations using Python and OpenCV.
- Generate reproducible visual outputs and record relevant parameters.
- Evaluate the method using the metrics and evidence specified in the assessment.
- Discuss limitations, failure conditions and suitability for the target application.

4. Software and Hardware Requirements

Software: Python 3, OpenCV-Python, NumPy, Pandas, Matplotlib and scikit-learn where classification or metric evaluation is required. Hardware: a computer capable of running Python, sufficient RAM for the image set, and a camera or documented image dataset appropriate to the experiment.

5. Application Context and Input Requirements

GreenHarvest Cooperative moves mangoes at 0.5 m/s beneath a camera. The system must measure fruit size, classify ripeness from peel colour and detect defects at least 2 mm wide across a 400 mm field of view. The decision limit is 1.2 seconds per fruit. The assessment requires at least 30 documented images or parameterized synthetic samples.

The selected technique should be judged against the constraints of the scenario. Relevant considerations include accuracy, sensitivity to lighting and viewpoint, computational time, memory use, ease of implementation and failure handling. A method that performs well on clean examples may be unsuitable when glare, blur, occlusion or uneven illumination is present.

TASK 9 – DETAILED THEORY

6. Introduction

An area-scan camera captures a complete frame. A rolling-shutter sensor exposes different rows at different times, potentially distorting moving objects. A line-scan camera captures one row at a time and constructs an image as the object moves. Correct line rate and synchronization are essential.

The method is useful because it converts raw visual data into information that can support the application decision. In a practical deployment, the algorithm must be considered together with camera properties, acquisition conditions, object variation and the quality of the reference annotations.

6.1 Fundamental Concepts

- Moving mango → area-scan frame / rolling-shutter rows / line-scan rows → reconstructed image
- Line rate is set by conveyor speed and required physical line spacing.
- A 100 Hz light modulation can create periodic row or frame intensity variation.
- Compare blur, skew, flicker sensitivity, data rate and runtime.

6.2 Mathematical Formulation

Line rate (lines/s) = conveyor speed (mm/s) / line spacing (mm).

At 500 mm/s and 0.5 mm line spacing, required rate = 1000 lines/s.

Data rate = width × line rate × channels × bytes/sample.

6.3 Assumptions and Design Considerations

- Input images are readable and have a documented colour order and data type.
- Annotations correspond to the same image coordinate system as the evaluated image.
- The evaluation split is fixed before parameter tuning.
- All compared techniques use identical input samples and metric definitions.
- Physical dimensions require calibration; pixel counts alone do not establish millimetres.

TASK 9 – METHODOLOGY AND ALGORITHM

7. Proposed Processing Workflow

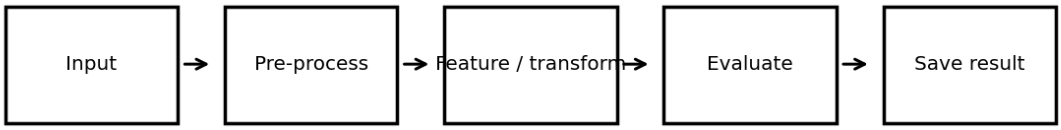


Figure 9.1 Processing workflow: Image Acquisition Arrangements

Figure 9.1: Conceptual workflow for Task 9.

7.1 Step-by-Step Procedure

- Prepare the input folder and verify that each file can be read.
- Load the image and annotation using the documented sample identifier.
- Convert the image into the representation required by the algorithm.
- Apply the candidate method using the recorded parameter values.
- Save the intermediate and final output images in the task Output folder.
- Compare the output with the reference annotation or measurement.
- Export the observation table as CSV and preserve the run configuration.

7.2 Algorithm

START → read configuration → load image and ground truth → validate dimensions → preprocess → apply selected operation → calculate metrics → save figures and CSV → record runtime → END.

7.3 Parameter Selection

Parameter selection should be conducted on a validation set. Begin with a documented baseline, vary one parameter at a time where possible, and record the effect on both visual quality and the primary evaluation metric. Avoid selecting a parameter solely because one example appears visually attractive.

TASK 9 – PYTHON IMPLEMENTATION

8. Program

The following program is a focused implementation template for the central operation. Update the file paths and dataset manifest to match the actual assessment dataset. The program deliberately raises an error when required files are missing instead of silently creating fabricated measurements.

```
import cv2, numpy as np
im=cv2.imread("mango.jpg")
h,w=im.shape[:2]
speed_mm_s=500; spacing_mm=.5
line_rate=speed_mm_s/spacing_mm
rolling=im.copy()
for y in range(h):
    rolling[y]=np.roll(im[y],int(8*y/h),axis=0)
cv2.imwrite("rolling_shutter.png",rolling)
```

```
mod=im.astype(np.float32)
for y in range(h):
   mod[y] *= 1 + .1*np.sin(2*np.pi*100*y/line_rate)
cv2.imwrite("modulated.png",np.uint8(np.clip(mod,0,255)))
print("line_rate",line_rate)
```

8.1 Program Explanation

- The input-loading stage reads the source image and checks that loading succeeded.
- The processing stage applies the method described in the theory section.
- The output stage writes the processed image or numerical result to disk.
- The evaluation stage must use the supplied annotation or reference data.
- The saved CSV/JSON record makes the experiment auditable and repeatable.

8.2 Execution Procedure

Install the required libraries, place the documented images and annotations in the expected folders, update the manifest or paths, and run the task program from its directory. Confirm that every expected output file is created. Record any warning, missing sample or execution failure in the observation notes.

TASK 9 – OUTPUT, OBSERVATION AND CONCLUSION

9. Output and Visual Evidence

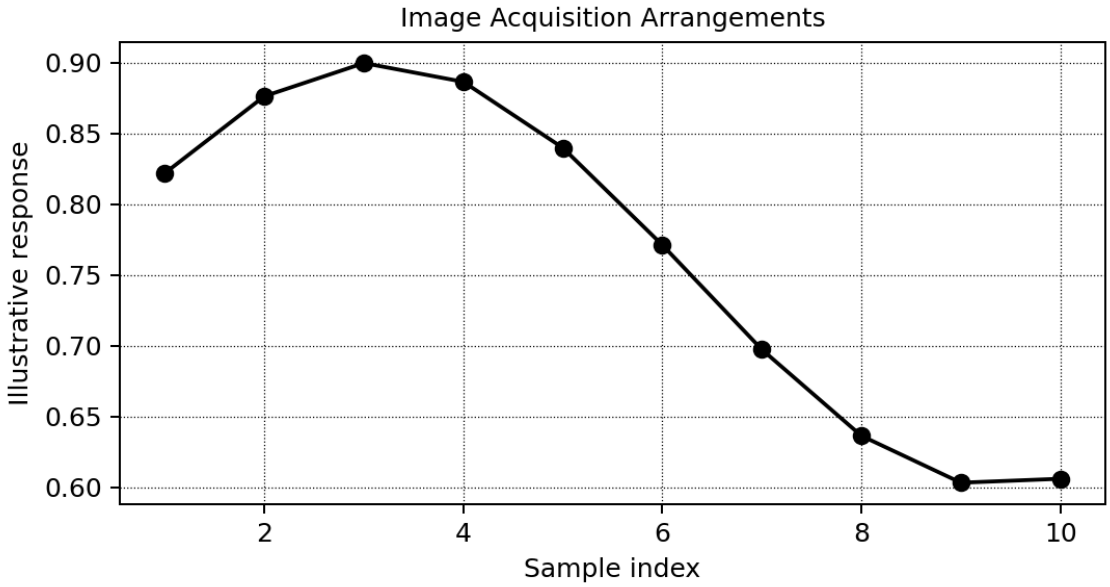


Figure 9.2: Conceptual response plot for Task 9. This figure illustrates the method and is not a measured dataset result.

10. Observation Table

Observation / parameter	Value to record	Evidence source
Input sample count	Enter actual number	Dataset manifest
Image size and data type	Enter actual dimensions/dtype	Program output
Algorithm parameters	Enter exact settings	Configuration file
Primary metric	Enter measured value	Evaluation CSV
Average latency	Enter measured seconds	Runtime log
Failure cases	Describe observed examples	Saved output crops

11. Result Analysis and Discussion

Interpret the completed observation table by identifying the strongest method on the held-out test set, the remaining errors, and whether the measured behaviour agrees with theory. Discuss input quality, parameter choice, annotation uncertainty and computational cost.

Do not invent accuracy, recall, PSNR, SSIM, corner-error or runtime values. If required input evidence is missing, write 'Not measured – input evidence required' and identify the missing image, mask, label or calibration target.

13. Conclusion

The experiment studied image acquisition arrangements in the context of the assessment scenario. The method, equations, implementation procedure and evaluation plan have been documented. The final engineering decision must be completed using the measured outputs from the supplied dataset. The selected approach should satisfy the application constraints while preserving a clear record of assumptions and failure cases.

TASK 10 – INTEGRATED FRONT END AND DESIGN DECISION

1. Experiment Number and Title

Task 10: Integrated Front End and Design Decision.

2. Aim

Combine acquisition, colour conversion, segmentation and defect measurement; export per-fruit results; compare three configurations using latency and recall; generate JSON configuration and code-produced report.

3. Objectives

- Understand the principle and purpose of integrated front end and design decision.
- Implement the required computer-vision operations using Python and OpenCV.
- Generate reproducible visual outputs and record relevant parameters.
- Evaluate the method using the metrics and evidence specified in the assessment.
- Discuss limitations, failure conditions and suitability for the target application.

4. Software and Hardware Requirements

Software: Python 3, OpenCV-Python, NumPy, Pandas, Matplotlib and scikit-learn where classification or metric evaluation is required. Hardware: a computer capable of running Python, sufficient RAM for the image set, and a camera or documented image dataset appropriate to the experiment.

5. Application Context and Input Requirements

GreenHarvest Cooperative moves mangoes at 0.5 m/s beneath a camera. The system must measure fruit size, classify ripeness from peel colour and detect defects at least 2 mm wide across a 400 mm field of view. The decision limit is 1.2 seconds per fruit. The assessment requires at least 30 documented images or parameterized synthetic samples.

The present experiment is part of the design of a computer-vision solution for the stated application. The central engineering question is not merely whether an algorithm can produce an output, but whether the output is repeatable, measurable and suitable for the operating conditions. The experiment therefore separates input preparation, processing, evaluation and interpretation.

TASK 10 – DETAILED THEORY

6. Introduction

An integrated inspection system must connect image acquisition to measurable grading outputs. The pipeline includes preprocessing, colour conversion, segmentation, size estimation, defect measurement, grading and report generation. A fixed test set is necessary to compare configurations fairly.

The method is useful because it converts raw visual data into information that can support the application decision. In a practical deployment, the algorithm must be considered together with camera properties, acquisition conditions, object variation and the quality of the reference annotations.

6.1 Fundamental Concepts

- Acquisition → preprocessing → colour representation → segmentation → size/defect features → grading → CSV/JSON
- Record fruit ID, width, ripeness, defect count, latency and grade.
- Compare three configurations on the same fixed test split.
- Treat uncertain cases as manual-review candidates rather than forcing a grade.

6.2 Mathematical Formulation

Mean latency = $\Sigma t_i / N$.

Throughput = 1/mean latency.

8 fruits/s requires mean latency ≤ 0.125 s.

Recall = TP/(TP+FN).

6.3 Assumptions and Design Considerations

- Input images are readable and have a documented colour order and data type.
- Annotations correspond to the same image coordinate system as the evaluated image.
- The evaluation split is fixed before parameter tuning.
- All compared techniques use identical input samples and metric definitions.
- Physical dimensions require calibration; pixel counts alone do not establish millimetres.

TASK 10 – METHODOLOGY AND ALGORITHM

7. Proposed Processing Workflow

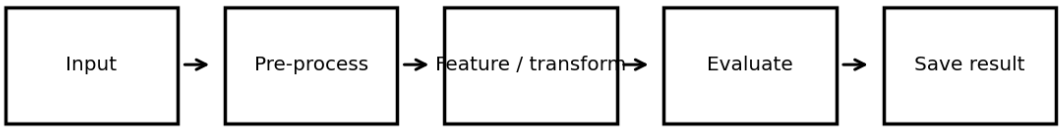


Figure 10.1 Processing workflow: Integrated Front End and Design Decision

Figure 10.1: Conceptual workflow for Task 10.

7.1 Step-by-Step Procedure

- Prepare the input folder and verify that each file can be read.
- Load the image and annotation using the documented sample identifier.

- Convert the image into the representation required by the algorithm.
- Apply the candidate method using the recorded parameter values.
- Save the intermediate and final output images in the task Output folder.
- Compare the output with the reference annotation or measurement.
- Export the observation table as CSV and preserve the run configuration.

7.2 Algorithm

START → read configuration → load image and ground truth → validate dimensions → preprocess → apply selected operation → calculate metrics → save figures and CSV → record runtime → END.

7.3 Parameter Selection

Parameter selection should be conducted on a validation set. Begin with a documented baseline, vary one parameter at a time where possible, and record the effect on both visual quality and the primary evaluation metric. Avoid selecting a parameter solely because one example appears visually attractive.

TASK 10 – PYTHON IMPLEMENTATION

8. Program

The following program is a focused implementation template for the central operation. Update the file paths and dataset manifest to match the actual assessment dataset. The program deliberately raises an error when required files are missing instead of silently creating fabricated measurements.

```
import cv2, time, json, pandas as pd
def inspect_fruit(path, cfg):
    image=cv2.imread(str(path))
    if image is None: raise ValueError(path)
    start=time.perf_counter()
    hsv=cv2.cvtColor(image,cv2.COLOR_BGR2HSV)
    mask=cv2.inRange(hsv,tuple(cfg["lower"]),tuple(cfg["upper"]))
    count,labels,stats,centroids=cv2.connectedComponentsWithStats(mask)
    latency=time.perf_counter()-start
    return {"fruit_id":str(path),"defect_components":count-1,
            "latency_s":latency}
config={"lower":[20,40,40],"upper":[90,255,255]}
with open("deployment.json","w") as f: json.dump(config,f,indent=2)
```

8.1 Program Explanation

- The input-loading stage reads the source image and checks that loading succeeded.
- The processing stage applies the method described in the theory section.
- The output stage writes the processed image or numerical result to disk.
- The evaluation stage must use the supplied annotation or reference data.
- The saved CSV/JSON record makes the experiment auditable and repeatable.

8.2 Execution Procedure

Install the required libraries, place the documented images and annotations in the expected folders, update the manifest or paths, and run the task program from its directory. Confirm that every expected output file is created. Record any warning, missing sample or execution failure in the observation notes.

TASK 10 – OUTPUT, OBSERVATION AND CONCLUSION

9. Output and Visual Evidence

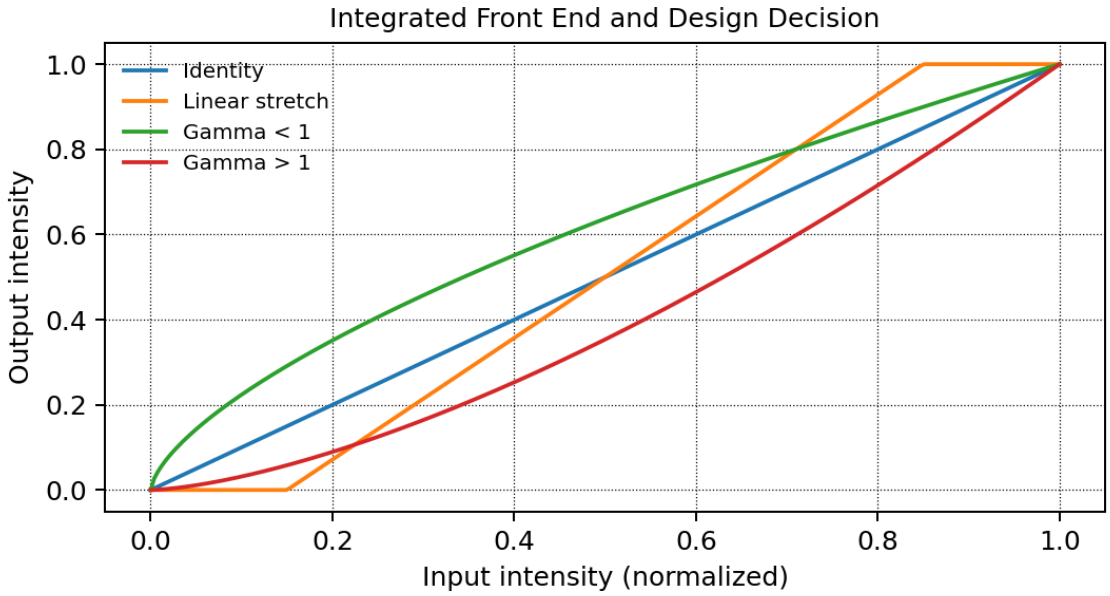


Figure 10.2: Conceptual response plot for Task 10. This figure illustrates the method and is not a measured dataset result.

10. Observation Table

Observation / parameter	Value to record	Evidence source
Input sample count	Enter actual number	Dataset manifest
Image size and data type	Enter actual dimensions/dtype	Program output
Algorithm parameters	Enter exact settings	Configuration file
Primary metric	Enter measured value	Evaluation CSV
Average latency	Enter measured seconds	Runtime log
Failure cases	Describe observed examples	Saved output crops

11. Result Analysis and Discussion

Interpret the completed observation table by identifying the strongest method on the held-out test set, the remaining errors, and whether the measured behaviour agrees with theory. Discuss input quality, parameter choice, annotation uncertainty and computational cost.

Do not invent accuracy, recall, PSNR, SSIM, corner-error or runtime values. If required input evidence is missing, write 'Not measured – input evidence required' and identify the missing image, mask, label or calibration target.

13. Conclusion

The experiment studied integrated front end and design decision in the context of the assessment scenario. The method, equations, implementation procedure and evaluation plan have been documented. The final engineering decision must be completed using the measured outputs from the supplied dataset. The selected approach should satisfy the application constraints while preserving a clear record of assumptions and failure cases.